



Operit2 Technical Whitepaper

A Persistent Agent Core for Personal Device Spaces

First Edition

Author Operit Team

September 2026



Abstract

Operit2 addresses an everyday problem: people use more and more devices, while capabilities are scattered across different endpoints. An Agent's context, actions, and results are often confined to one machine or one session. We want to connect phones, desktops, browsers, and cloud devices into a personal device space owned by the user. Each device retains its own system capabilities and permission boundaries, while the Agent carries tasks, context, and user intent across devices.

This whitepaper records Operit2's engineering judgments and long-term direction. We care about whether a user's conversations, tasks, data, device relationships, and authorizations can persist as models, prompts, tool planning, and interaction patterns continue to change. To that end, we want model adaptation and the runtime to remain replaceable, while node identity, Host capabilities, persisted records, synchronization rules, and user control are maintained as long-lived foundations.

Operit2 is currently in preview. This document uses the existing source tree and architecture documents as its engineering baseline, and explicitly distinguishes the current foundation from the direction of evolution: node connections, persistent synchronization, and runtime behavior already have implementations; capability scheduling, cross-node plugin continuation, and a complete recovery experience still need work. Research citations explain the design rationale; they are not performance validation or industry comparisons.

Contents

- 1 Starting Point and Problem Reframing
- 2 A Personal Agent Device Space
- 3 Operit2 Engineering Architecture
- 4 Conversation Sovereignty, Task Execution, and Context Continuity
- 5 Device Capabilities, Health, and Scheduling
- 6 Permissions, Synchronization, and User Sovereignty
- 7 A Pluggable, Evolvable Runtime
- 8 Research Context and Technical Position
- 9 Reliability, Migration, and Preview-Stage Evolution
- 10 Implementation Priorities and Closing
- Appendix Source Baseline and References



Starting Point and Problem Reframing

Operit's first-generation product started as an Android Agent. The team did not treat the phone as a thin client responsible only for displaying a conversation; it brought a local Linux execution environment, terminal sessions, a file system, and mobile permission capabilities into the Agent's working scope. The historical code includes a PRoot-driven Ubuntu environment, visible PTY sessions, background shell reuse, and an implementation that exposes the Ubuntu file system through Android's Storage Access Framework. This experience confronted us early with a practical reality: a device's form should not determine its value in the Agent world. [2]

After building deeper mobile Agent experience, Operit2 initially focused on cross-platform expansion. But putting the same conversation and tools on more endpoints does not by itself solve the fragmentation users experience: phones are often carried everywhere but are not suited to every heavy task; desktops are good for creation and review but may not always be available; cloud devices can stay online, but should not receive identity privileges simply because of that. We gradually shifted the focus to one question: how can an Agent's tasks and context continue across one person's devices?

This reframing also defines Operit2's product scope. We prioritize serving individuals rather than turning device collaboration into an organizational governance system. One person may own multiple devices, and may add a cloud server, browser, or newly purchased computer to their space; but device collaboration is not a multi-person approval chain, enterprise organization structure, or management hierarchy. Operit2 aims to lower the operational barrier to accessing AI capabilities, so complex device capabilities can be understood, authorized, and scheduled, rather than requiring every user to first learn terminals, environment variables, and deployment topologies.

A Personal Agent Device Space

We call one instance running Operit Core a CoreNode. It can be hosted by a supported mobile, desktop, or cloud runtime; browser runtime capabilities and access patterns need to be distinguished separately. Each node uses its own HostManager to access the files, terminal, browser, network, model, storage, and platform-interaction capabilities provided by its local Host. A node's collaboration role should be determined by availability, capability, and user policy, rather than being predetermined by operating system or device category.



Figure 1. Collaboration among equal CoreNodes in a personal Space

Space is a collaboration scope made up of multiple CoreNodes that have established trust. It first determines which nodes share continuously synchronized business records and which nodes can carry routed traffic. It is not a central server, nor does it directly expose a remote file system, administrator privileges, or running processes to other nodes. A node's local capabilities remain managed by its own Host; cross-node collaboration means that the right node executes in the right context, not that every capability is copied to the initiator.

Concept	Role in a personal space	Boundary that must hold
CoreNode	Independent node running Core and a local Host; can handle conversations, tools, or ongoing tasks.	Nodes remain equal; being online for longer does not make a node central.
Host	Provides local files, terminal, browser, network, model, and system interfaces.	System permissions are determined by the local machine and deployment environment; they are not inherited through synchronization.
Space	Organizes members, the scope of persistent synchronization, and routable collaboration relationships.	It does not replace local execution environments or imply shared root privileges.
Link and PeerLink	Handles pairing, authentication, real-time calls, observation streams, and multi-hop forwarding.	Pairwise trust is the foundation of direct network connections.
Binding	Records which node should continue the next segment of executable work.	Changing execution location does not turn chat or Agent data into a single point of ownership.

Operit2's Engineering Architecture

Operit2 is built on a Rust Core and connects through platform Hosts, Flutter applications, the CLI, and Web Access. The Core workspace separates responsibilities across the Host API, base models, persistent storage, tools, plugins, model providers, the runtime, node routing, Access, Link, and agent generation. Some crate boundaries and lifecycle management are still being refined. Upper runtimes assemble the implementations, while lower layers expose capabilities through data structures and traits, reducing coupling between platform adaptation, model integrations, and front ends. [1]



Different modules need different evolution speeds. Model providers, prompts, tool planning, UI, and script runtimes change quickly; Host APIs, storage boundaries, node identity, synchronization records, and permission semantics require careful maintenance. Core places the Host API, data models, and storage in lower layers, then composes tools, providers, the JavaScript bridge, and application runtimes, leaving room to replace models, add platforms, or adjust interaction patterns.

Persistent Synchronization and Real-Time Transport

In node collaboration, Operit2 intentionally separates two data paths. SpacePersistenceSyncService subscribes to local persistence changes and exchanges synchronization operations, vector clocks, and necessary content-addressed files between directly paired nodes. Chats, preferences, object data, Bindings, and runtime files can be synchronized according to their declared persistence scope. Node identity private keys, pairwise pairing keys, HostManager platform state, the local listening address, and temporary interaction handles remain local. This separation makes recovery and continuation a designable problem, and prevents transient connection state from being mistaken for replicable business fact.

The real-time path carries Core calls, watches, pushes, stream events, Agent continuations, and responses from Host interactions. CoreNodeRouter uses routing metadata to decide whether a request executes locally or reaches a target CoreNode through PeerLink. Real-time transport may wait until persistent synchronization reaches a certain clock position, but that barrier only guarantees that the target node sees enough task facts; it does not mean an in-flight model request, PTY, browser session, or process handle is migrated unchanged.

Host as the Capability Boundary

HostManager exposes file systems, terminals, HTTP, browser automation, browser sessions, storage, key custody, runtime scheduling, and local inference to Core through traits. HostEnvironmentDescriptor also describes the platform, current permissions, isolation state, structured capabilities, and workspace root. Windows, Linux, Android, OpenHarmony, Apple, and Web Hosts can expose different capabilities through this boundary, while the upper-level Agent does not need to hard-code platform differences into conversation logic.

In Operit2, a browser is not merely a page view. Web Access can serve as a browser-facing surface for an already-running node; a browser-platform Host has its own runtime and capability description. The two must not be conflated. The former lets users approach their Agent from a browser; the latter leaves an engineering path for a browser to become an independent running node.

Conversation Sovereignty, Task Execution, and Context Continuity

If multiple devices inject live input into the same model context at the same time, task order and execution ownership can easily become confused. Operit2 separates the main conversation from execution: the main conversation retains the interaction ownership of the initiating device; execution nodes handle local tool loops for tasks or subtasks; Space asynchronously synchronizes state, results, artifacts, and recoverable records. Other devices can view progress, but synchronization does not require every process to be injected into the same model request in real time; offline nodes must also catch up after reconnecting.

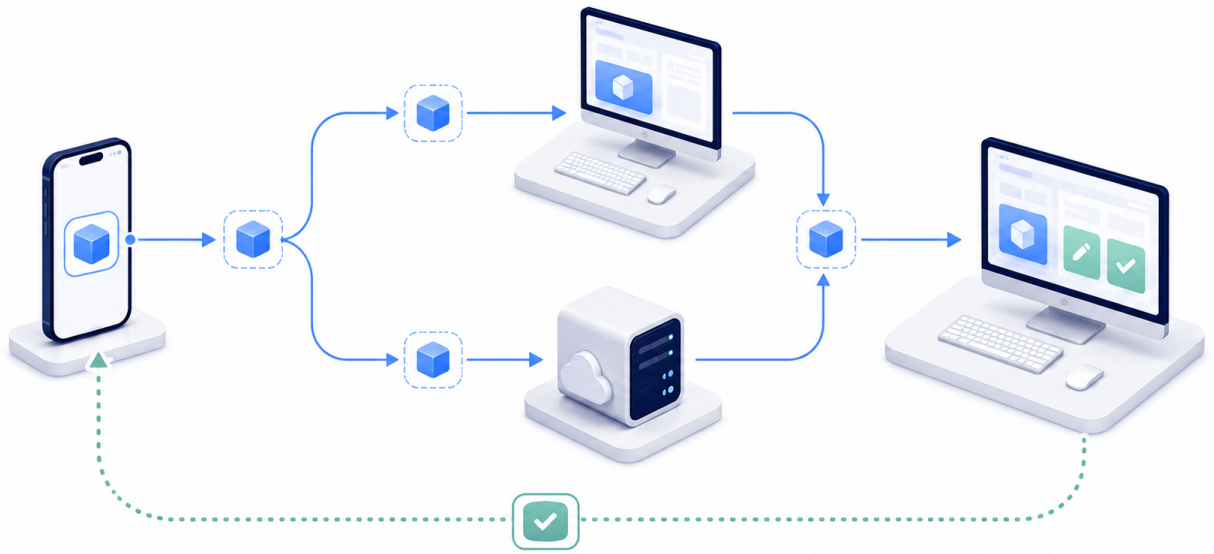


Figure 2. Continuity across task initiation, execution, synchronization, and review

Binding records which CoreNode should continue the next segment of work, and uses an incrementing generation to identify changes in execution ownership. Handoff is selected at the boundary where the tool result has been persisted but the next model request has not started; after obtaining the necessary records, the target node continues and the old node should stop advancing. This mechanism constrains the executor for one round, but cannot by itself guarantee that an external action occurs only once. Retries after network failures still require idempotency, deduplication, and result verification to avoid duplicate submissions.

Recoverable Task Facts from Transient Runtime

Shells, stream connections, browser sessions, and model requests contain many transient details. The current continuation mechanism does not migrate these running instances; it stores enough task facts to understand and continue the work: execution node, time, tool parameters, major actions, results, artifacts, and necessary error information. Stage summaries help the next device understand progress quickly, while raw logs and artifacts can be retained as detailed evidence according to policy. Summaries inevitably omit details, so recovery depends on the records actually saved; it cannot promise lossless recovery from a summary alone.

Information layer	Typical contents	Role in collaboration and recovery
Live runtime layer	Process handles, PTYs, stream connections, short-lived caches, UI state.	Managed by the current node; can end or be rebuilt when the connection is interrupted.
Task trajectory layer	Tool parameters, execution node, important events, results, errors, artifact references.	Can be synchronized, audited, and understood for continuation on another node.
Semantic history layer	Stage summaries, user intent, decision rationale, next steps that can continue.	Controls main-model context density; supports backup and recovery.
Local sensitive layer	Device private keys, pairing keys, temporary Host permissions and system handles.	Kept on the node; not synchronized by default.



Device Capabilities, Health, and Scheduling

A Linux cloud device is an ordinary CoreNode in a personal Space. It may take on more work because it stays online, has suitable compute, or provides favorable service conditions; that is a result of task assignment, not a central identity granted by the system. New computers, new servers, and independent browser nodes that join a Space in the future should follow the same principle: retain their own capabilities and participate according to task needs and user choice.

Capability scheduling remains a direction that needs design and validation. We plan to select nodes using three types of information: the platform, hardware, runtime, and services declared at join time; current reachability, resource pressure, and connection quality; and execution latency plus success and failure records accumulated through long-term use. One failure is only one piece of evidence and should not permanently reject a capability; newly added GPUs, installed services, or network changes should also update the capability assessment in time. The table below describes dimensions that scheduling must consider; it does not mean that a complete mechanism is already implemented.

Capability view	Question it answers	Scheduling implication
Declared capability	What platform, runtime, hardware, and services does this device claim to have?	Used for initial matching and compatibility filtering.
Real-time health	Is it reachable now, able to receive tasks, and under resource pressure?	Short-term availability; time-sensitive.
Long-term evidence	Has it completed this kind of work, and what are its latency and stability?	Accumulates evidence of actual capability and corrects assessment as the environment changes.
User policy	What are the preferences for privacy, cost, battery, preferred devices, cloud use, and redundancy?	Efficiency follows user choice.

We are considering redundancy for failover, controlled parallelism, and subtask splitting; specific policies remain to be validated. Parallel work must comply with permission, cost, and resource limits; tasks involving external writes especially need explicit execution ownership and handling for idempotency, deduplication, and failure verification. A lease or generation can form only one part of the mechanism; it cannot replace control over external side effects. The goal is for devices to participate when appropriate, not for every node to remain fully loaded.

Permissions, Synchronization, and User Sovereignty

The core of a personal Agent is not to make every decision on a person's behalf, but to let the person see, understand, and decide important boundaries. Operit2's permission architecture separates external isolation, capabilities granted to the Host by the system, the AI's current capability mode, and approval for a specific tool call from top to bottom. Lower layers may further restrict capabilities, but cannot bypass upper layers. Full permissions do not mean root or administrator escalation; if a device process lacks a system capability, the Agent cannot acquire it through an in-app switch.



Level	Responsibility	Meaning understandable to the user
Execution environment isolation	External boundary such as VMs, containers, system accounts, and Android app sandboxes.	What the device or deployment environment itself permits.
Host authorization	Actual capabilities granted to the current Host by the OS or deployment environment.	What this machine and account can really do.
AI capability mode	Runtime limits such as read-only, workspace read/write, or full permissions.	What the AI is allowed to read, write, and call at this moment.
Specific tool approval	Confirmation for one consequential tool call.	Which action will happen where, and whether I authorize it.

Sensitive synchronization still needs a better interaction model. We want users to see the objects to be synchronized, the target node, and the consequences before deciding whether to synchronize or keep them local; revoking authorization, rotating credentials, and recovering data should also have usable entry points. Revoking future access does not mean reclaiming data that has already been copied, and the system needs to explain the difference. Command lines and environment variables should not be prerequisites for understanding these choices; final authority always belongs to the user.

A Pluggable, Evolvable Runtime

Model adaptation, prompts, tool planning, UI, and executors may all change as model capabilities and usage patterns evolve. Operit2 already provides extension points such as ToolPkg, Skill, MCP, Compose DSL, the JavaScript bridge, and Wasm, and includes package resolution, lifecycle management, a plugin SDK, and TypeScript declarations for authors. These entry points provide a foundation for replacing and extending the runtime, but the interfaces are still evolving and not every module has been made pluggable. [1]

The premise for running plugins across devices is that they clearly declare the required runtime, Host capabilities, permissions, compatibility versions, state scope, and recovery information. We want compatible nodes to execute or continue tasks accordingly while preserving the boundaries of platform-specific capabilities. The table below proposes dimensions for the next SDK contract; it is not yet a stable specification. At present, one cannot assume that an arbitrary plugin will run on every device.

Suggested SDK contract dimension	Suggested description	Value across nodes
Identity and compatibility	Package identifier, version, source, compatibility range for Core and SDK.	Lets a node know whether it can parse, load, and migrate results.
Runtime and dependencies	JavaScript, Wasm, or another runtime; required services and resources.	Avoids disguising platform assumptions as general capability.
Capabilities and permissions	Host capabilities, tool effects, sensitive operations, and approval requirements.	Bases scheduling and user confirmation on real risk.
State and continuation	Local, task, or Space state; artifacts and recovery information.	Leaves a continuation trail across compatible nodes.



Research Context and Technical Position

Large language model-driven Agents typically combine perception, reasoning, action, and memory in a loop. The survey by Xi et al. describes LLM Agents as systems centered on a model and combined with perception and action; this is a useful framework for understanding Agents, but it does not replace the identity, permission, persistence, and device-collaboration problems that a product-grade runtime must solve. [6] Operit2's position is to treat these runtime problems as first-class engineering objects.

Action Trajectories and Verifiable Task Facts

ReAct shows that interleaving reasoning trajectories with external actions can help a model maintain a task plan and make the process easier to understand than a final answer alone. [3] Operit2 does not treat the model's hidden chain of thought as something that should be synchronized without limit, but it recognizes that task processes need verifiable external facts. Therefore, tool calls, inputs, results, errors, artifacts, execution nodes, and stage summaries hold an important place at the system level. They serve the current conversation, cross-device viewing, backup and recovery, and the next continuation.

Layering Model Context and External Memory

MemGPT connects limited context windows with hierarchical memory management, pointing out that long conversations and large documents need to select information across storage layers. [4] Generative Agents further demonstrates how experience records, reflection, and dynamic retrieval influence subsequent planning. [5] These studies inspire us to treat model context as scarce working space, not the sole container for a user's entire digital life. Operit2 does not claim to solve the model's endogenous long-term-memory problem; it addresses the continuity foundation outside the model: working state, persistent personal and task history, the state of the device world, permissions, and records of data ownership.

Accordingly, cross-device record storage and providing context to a model should be handled separately. The main model receives summaries, evidence, and accessible references as the current task requires; detailed logs and raw artifacts are read when needed. This design aims to reduce context noise while preserving traceability; omissions in summaries, retrieval quality, and continuation results still need validation through real tasks.

Who Still Needs What After Models Improve

Stronger models may reduce reliance on specialized prompts, fixed Agent loops, and hand-written tool planning. Operit2 needs to preserve replacement space for these changes. At the same time, whenever a task still executes on a device, the system must record accessible resources, authorization scope, task results, and the recovery path after a device change. The form of a future harness is uncertain; for now, we choose to maintain these runtime foundations that directly affect users.



Reliability, Migration, and Preview-Stage Evolution

Operit2 is in a stage where infrastructure refactoring and stability improvements proceed in parallel. We will not use "forever compatibility" to block necessary architectural improvements, nor treat every refactor as permission to silently lose user continuity. Interfaces, data formats, plugin contracts, and synchronization semantics should be versioned; when migration occurs, users should understand what is being migrated, retain recoverable records, and know whether task and device relationships still hold.

For personal users, migration should cover conversation and task history, necessary materials, extension state, and explainable task trajectories. Our goal is that when a new phone, computer, or server joins, a complete synchronized copy or backup should minimize manual transfer. Recovery still depends on replica scope and format compatibility; a new node needs to reconfirm identity, trust relationships, and local permissions. Runtime details that have been lost without a copy cannot be recovered from nothing.

Reliability needs to be measured through observable behavior. Reconnection and release of long-lived connections, cancellation propagation, backpressure, stream termination, task deduplication, content validation, failure handoff, execution generation, and recovery records are all runtime qualities that require long-term validation. Rust gives Core a language and ecosystem foundation suitable for long-running systems, but the language itself does not automatically guarantee the absence of resource leaks or lifecycle problems; final quality comes from explicit lifecycles, backpressure, cancellation, resource control, and observability.

Implementation Priorities and Closing

Next, the focus is to let users start, inspect, approve, continue, and recover Agent work on the appropriate device, reducing the burden of repeated configuration and context organization. We will fix issues and improve stability first, then gradually complete the following capabilities.

Priority	Real problem to solve	Expected foundation
Capability orchestration	Capabilities exposed across devices still need to become an understandable, selectable, executable toolchain.	Nodes selected based on contracts, health, historical evidence, and user policy.
Long-running quality	Continuity of tasks, streams, connections, cancellation, and resource release must withstand long-term use.	Task stability, observability, and recoverability verified systematically.
Migration and recovery	User continuity must not depend on manually moving data when devices join or leave.	Clear user paths for synchronization, backup, recovery, and selective reconstruction.
Plugin SDK	Extension authors need to know how to declare capabilities, risks, state, and compatibility.	Support execution or continuation on compatible nodes, and clarify incompatible conditions.
Human authorization UX	Confirmation, revocation, rotation, and policy selection should be natural interactions, not a command-line burden.	Ordinary users understand sensitive synchronization and side-effecting operations.

We hope each device can retain what it does best, so that when users switch devices they can keep doing their work. Phones are convenient for communicating anywhere; desktops suit creation and review; nodes that stay online can keep tasks running. Operit2 will continue to undergo necessary refactors. We want to invest in clear permissions, reliable records, and recoverable collaboration, making Agents easier for ordinary people to use.



Appendix Source Baseline and References

Source Baseline

The table below preserves the source and documentation entry points compiled for the first edition on September 7, 2026, so readers can inspect the engineering basis. This document also contains design goals; directory locations and interfaces may change with refactoring and do not constitute an API stability commitment.

Topic	Source or documentation entry	Content used for cross-checking
CoreNode Space Binding	<code>docs/core-node-space-binding-architecture.md</code> <code>core/crates/node/runtime/src/CoreNodeRouter.rs</code>	Node equality, Space scope, Binding routing, and single-executor semantics.
Persistent synchronization	<code>core/crates/node/runtime/src/SpacePersistenceSyncService.rs</code>	Change-triggered synchronization, directly paired nodes, synchronization operations, and file transfer.
Link and Access	<code>docs/link-access-architecture.md</code>	Separation of responsibilities between local application links and Space node links.
Host capabilities	<code>core/crates/foundation/host-api/src/HostManager.rs</code> <code>core/crates/foundation/host-api/src/lib.rs</code>	HostManager, environment descriptors, platform capabilities, and runtime traits.
Permission boundaries	<code>docs/permission-access-architecture.md</code>	Layering of execution environment, Host authorization, AI capability modes, and tool approval.
Plugin runtime	<code>core/CRATE_BOUNDARIES.md</code> <code>plugins/</code> <code>core/crates/plugin/sdk/</code>	Boundaries for ToolPkg, JavaScript, Wasm, SDK, and runtimes.
Historical starting point	Operit repository's terminal module <code>TerminalManager.kt</code> <code>LocalTerminalProvider.kt</code> <code>UbuntuDocumentsProvider.kt</code>	Ubuntu PRoot, terminal sessions, and SAF file-system exposure on Android.



References

- [1] AAswordman. Operit2 source repository. GitHub. <https://github.com/AAswordman/Operit2>. Accessed September 7, 2026.
- [2] AAswordman. Operit source repository. GitHub. <https://github.com/AAswordman/Operit>. Accessed September 7, 2026.
- [3] Yao, S., Zhao, J., Yu, D., et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. <https://arxiv.org/abs/2210.03629>
- [4] Packer, C., Fang, V., Patil, S. G., et al. MemGPT: Towards LLMs as Operating Systems. arXiv 2023. <https://arxiv.org/abs/2310.08560>
- [5] Park, J. S., O'Brien, J. C., Cai, C. J., et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. <https://arxiv.org/abs/2304.03442>
- [6] Xi, Z., Chen, W., Guo, X., et al. The Rise and Potential of Large Language Model-Based Agents: A Survey. arXiv 2023. <https://arxiv.org/abs/2309.07864>